Evaluating Learned Spatial Indexes

Sachith Pai
University of Helsinki
Helsinki, Finland
sachith.pai@helsinki.fi

Jun Yang
University of Helsinki
Helsinki, Finland
jun.yang@helsinki.fi

Michael Mathioudakis
University of Helsinki
Helsinki, Finland
michael.mathioudakis@helsinki.fi

arXiv:2606.19034v1 [cs.DB] 17 Jun 2026

Abstract

Learned indexes improve query performance by adapting search structures to data and workload distributions. Although many learned indexes have been proposed, their trade-offs remain insufficiently understood for spatial range queries, where performance depends not only on model accuracy but also on data and query skew, layout granularity, selectivity, and storage behavior.

In this work, we perform an experimental study of learned indexes for spatial range queries. We examine a representative set of indexes and address seven fundamental questions: (1) How does block size influence query latency, and what configurations yield optimal performance under varying selectivities? (2) How do skewed data and query distributions impact index performance? (3) How do indexes balance refinement and scan costs, and which designs favor one over the other? (4) How do disk-based storage conditions alter optimal block size and latency trade-offs compared to in-memory settings? (5) What are the construction costs of different indexes, and under what query volumes are these costs amortized? (6) For a given data and query workload, which index is expected to perform best? (7) Do index-selection insights learned from synthetic data generalize to real-world data distributions? To enable the analysis, we use a framework with a common storage backend, standardized query execution pipelines, and controlled variations in data and query skew.

Our experiments reveal critical insights into refinement vs. scan trade-offs, the impact of block size, and the interplay between selectivity and layout effectiveness. We synthesize these findings into a workload-based decision tree for index selection and validate it on real OpenStreetMap point sets with synthetic queries, confirming that its recommendations exhibit minimal decision regret and typically yield near-optimal query performance.

1 Introduction

Indexes are essential in database management systems (DBMSs), enabling efficient data retrieval for specific queries. Because of their central role in performance, indexing methods are extensively studied, and new approaches continue to emerge. Recently, advances in machine learning (ML) have inspired a new class of data-aware indexing techniques. These methods, known as learned indexes [14], use compact predictive models to approximate data distributions or query costs, thereby tailoring the index structure to the underlying dataset. Over the past few years, learned indexes have been proposed for diverse tasks, including one-dimensional search, spatial indexing, and nearest-neighbor queries.

Despite growing interest in learned indexes, comprehensive and systematic benchmarking remains limited [13, 20]. Most existing studies introduce new learned indexing techniques and demonstrate improvements over selected baselines in narrowly defined scenarios. However, the performance of learned indexes can vary significantly depending on factors such as data distribution, query workload, index granularity, and storage medium. To draw reliable conclusions, we need careful comparisons under controlled conditions that reveal when learned indexes truly outperform traditional ones. Such evaluations are essential for guiding future designs and enabling practical adoption. While benchmarking frameworks exist for one-dimensional indexes, equivalent efforts for two-dimensional spatial indexing remain largely absent.

Spatial range queries introduce additional sources of variation because an index must both identify relevant spatial regions and control the amount of irrelevant data scanned within them, especially when data and query distributions are skewed. As a result, comparisons depend not only on model accuracy, but also on data skew, query skew, layout granularity, query selectivity, and storage behavior. This study systematically benchmarks learned spatial indexes for static two-dimensional point data under both in-memory and disk-backed range-query execution. We evaluate a curated set of learned and traditional indexes within a unified benchmarking framework that uses a common storage backend, controlled synthetic workloads, and real OpenStreetMap validation data. We focus on range queries, as they are the core operation for spatial indexes. Our central thesis is that a rigorous comparative framework can reveal how learned and traditional indexes perform under different data and workload conditions for spatial range queries. To make this comparison actionable, we also evaluate whether empirical findings can be translated into low-regret index choices, where a recommendation remains useful even if it does not always identify the single fastest index. Our main contributions are:

(1) We analyze how block size influences performance and evaluate the impact of data and query skew on index performance.
(2) We analyze query latency by decomposing it into the refinement phase (identifying blocks that may contain matching points) and the scan phase (checking points within the identified blocks).
(3) We examine index performance with disk-based storage.
(4) We study when index construction costs are offset by improved query performance.
(5) We synthesize findings into a decision tree that guides index selection based on data and query workload characteristics.
(6) We validate the index-selection guidance on real-world OpenStreetMap point sets paired with synthetic query workloads.

2 Related Work

Initial research on learned indexes concentrated primarily on the one-dimensional (1D) indexing problem. As several learned methods emerged, each using distinct modeling approaches, there was

a growing need for systematic evaluation. This led to the development of multiple benchmark studies. The SOSD benchmark [13] provided a foundational evaluation framework for learned indexes, comparing a range of indexes across diverse synthetic and real-world datasets. Later studies expanded this scope by introducing comprehensive experimental evaluations that considered a wide range of learned indexes, data distributions, query workloads, and index update mechanisms [22, 32]. Other works conducted targeted analyses of specific index families, particularly recursive model indexes, revealing performance bottlenecks in scalability and update efficiency [21]. Several evaluations examined the readiness of updatable learned indexes, identifying limitations in their ability to handle dynamic workloads efficiently [9, 37]. Follow-up investigations explored the integration of updatable indexes into disk-resident database systems, revealing critical design trade-offs in in-memory layout and I/O behavior [15]. Finally, recent work has examined the feasibility of efficiently constructing learned indexes, proposing sampling-based techniques to reduce training cost while preserving accuracy [6]. These studies have collectively advanced the understanding of 1D learned indexes and enabled detailed comparisons across diverse operational settings.

However, most of these evaluations are limited to the 1D setting, with relatively little attention given to learned indexing in higher-dimensional or spatial contexts. To our knowledge, there has been only one experimental survey, conducted by Liu et al. [20], which analyzed a collection of ten multidimensional learned indexes (including ZMIndex, RSMI, FLOOD, and LISA [17]) on point-query workloads. Their study finds that learned indexes often outperform classical indexes for in-memory range queries on points: for example, FLOOD and LISA were faster than R-tree variants across a variety of query selectivities, achieving notable speedups on large point sets. Learned indexes also tend to use space more efficiently, yielding smaller memory footprints than R-tree indexes by storing model parameters instead of large node hierarchies. However, the results also revealed some limitations. Some methods, such as RSMI, incur prohibitively high training costs at large scales, raising concerns about scalability. Furthermore, for query types such as k-nearest neighbor (k-NN) search, learned indexes often fail to consistently outperform well-optimized traditional methods, such as KDTREE. This mainly stems from the iterative probing required for k-NN queries, which reduces the benefit of learned predictions. Overall, while learned indexes are well-suited for batched range queries on static point data, their performance advantage is not universal.

Figure 1 summarizes this gap by showing which index pairs have been compared in prior learned-spatial-index studies and which comparisons remain missing. While our work is similar in scope to Liu et al. [20], it differs in five crucial ways that address previously unexamined benchmarking shortcomings in spatial learned indexing: (1) We re-implement the methods evaluated in our experiments to avoid implementation-induced biases, whereas their analysis used original implementations and open-source libraries. (2) We analyze all methods within a unified framework, which stores data points in blocks with a common structure across indexes. (3) We construct diverse synthetic datasets and query workloads to systematically evaluate index behavior under varying levels of skew and selectivity, an aspect not addressed in previous work. (4) We present

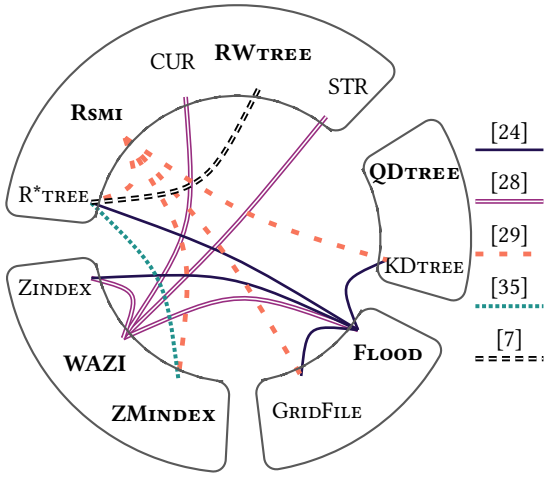


Figure 1: Chord diagram showing the indexes evaluated in this work (nodes in the diagram) and prior comparisons between them (edges). Learned indexes are shown in bold. Our work aims to fill the missing comparisons in the analysis of learned spatial indexes.

detailed analyses of performance-affecting factors not covered in previous studies, including block size (Section 5.1), the breakdown of latency into refinement and scan phases (Section 5.3), effects of data and query distributions (Section 5.2), and performance on disk-based data (Section 5.4). (5) We synthesize our empirical findings into a decision tree (Figure 10) that provides practical guidance for selecting the most suitable index under given workload characteristics. Together, these differences allow this work to address important questions about learned spatial indexes that have not been previously examined.

3 Compared Indexes

We evaluate both traditional and learned indexes, grouped by design principle into Grid-based Partitioning Indexes, Space Partitioning Indexes, Data Partitioning Indexes, and Ordering-based Indexes. The following sections describe each method, its implementation choices, and any learned components used. Table 1 summarizes the key characteristics of each index.

3.1 Grid-based Partitioning Indexes

Grid-based Partitioning Indexes divide the data space into a regular grid of cells. Each point is assigned directly to the cell containing it.

3.1.1 GRIDFILE [26]. The GRIDFILE partitions the search space with a fixed grid layout. Points are grouped into blocks by their grid cells, and the number of splits per dimension is chosen so that blocks contain the required average number of points.

3.1.2 FLOOD [24]. The FLOOD index uses an optimized grid layout that minimizes query latency for a given data distribution and query workload. It applies surrogate-based optimization [25, 30], training a surrogate ML model on measured query latencies for a small set of grid layouts to predict the latency of unseen layouts. These predictions serve as the objective for evaluating candidate layouts.

Index	Query-Aware	Learned Component
GridFile [26]	No	-
Flood [24]	Yes	Surrogate latency predictor.
KDtree [5]	No	-
QDtree [38]	Yes	RL-agent for data splitting.
STR [16]	No	-
R*tree [3]	No	-
Rsmi [29]	No	Neural network at each node.
CUR [31]	Yes	-
RWtree [7]	Yes	Learned 4-D MBR-overlap cost estimator.
Zindex [2, 23, 34]	No	-
ZMindex [35]	No	Learned 1-D index (RMI).
WAZI [28]	Yes	Data/query density estimator.

Table 1: List of indexes compared, grouped into categories, with the learned indexes shown in bold.

3.1.3 Implementation. The Grid-based Partitioning Indexes are implemented as two-level search trees over vertical and horizontal partitioning boundaries. For GridFile, we choose equal vertical and horizontal splits independently to form rows and columns. Our Flood implementation uses the same structure, but directly evaluates candidate layouts on a data sample instead of training a surrogate model. We avoid surrogate optimization because, in our two-dimensional in-memory setting, surrogate training costs more than simulating candidate query latencies.

3.2 Space Partitioning Indexes

Space Partitioning Indexes recursively divide the data space into smaller subregions. Each node represents a domain region, producing a hierarchical spatial decomposition for index construction.

3.2.1 KDtree [5]. The KDtree recursively splits the data space along alternating dimensions using median splits, so the points in the current region are divided into two roughly equal subsets. This process continues until each cell has at most the target block size, producing a binary tree.

3.2.2 QDtree [38]. The QDtree index customizes KDtree by learning the split dimension and location to minimize block accesses. Its training algorithm uses a reinforcement learning agent to select these decisions at each node while constructing candidate trees. The completed tree is rewarded according to block-skipping efficiency, allowing the agent to learn better partitioning policies over time.

3.2.3 Implementation. The Space Partitioning Indexes are implemented as binary trees, with each node defined by a split dimension and split location. In the case of KDtree, the split dimension alternates at each level of the tree, and the split location is selected as the median value along the chosen dimension. QDtree is implemented as an optimized KDtree, using a sampling-based strategy to select both the split dimension and location while minimizing the specified objective function. We also experimented with the RL construction method from [38], which learns a split policy from a finite set of candidate split locations. In our implementation, the agent frequently appeared to settle in local optima, yielding policies close to fixed split rules rather than reliably data-dependent decisions. In our setting, the RL approach was also feasible only for shallow trees, since deeper trees require many candidate split locations and make training prohibitively expensive. We therefore use sampling-based optimization for QDtree.

3.3 Data Partitioning Indexes

Data Partitioning Indexes group points into subsets represented by minimum bounding rectangles (MBRs) arranged hierarchically. They reduce query cost by optimizing point partitions and MBR overlap. Classic R-trees [11] build these structures iteratively, inserting objects into leaf nodes while minimizing the enlargement of MBRs along the path. Below, we describe several variants and learned extensions.

3.3.1 STR [16]. The Sort-Tile Recursive (STR) algorithm bulk-loads R-trees by sorting points along one dimension, partitioning them into equal-sized tiles, and repeating along alternating dimensions until blocks reach the target size. This produces compact trees with low MBR overlap.

*3.3.2 R*tree [3].* The R*tree refines R-tree insertion by considering perimeter and perimeter change in addition to area enlargement [4]. This favors square-like MBRs, reducing node overlap and query cost.

3.3.3 Rsmi [29]. The Recursive Spatial Model Index (Rsmi) is a learned R-tree-style index. It recursively partitions spatial data into regions handled by learned models rather than explicit bounding boxes. Rsmi first maps data into rank space, where each dimension is replaced by rank order, then applies a space-filling curve to sort and group points into blocks. At each stage, a learned function routes points to sub-models, mirroring R-tree splits with neural networks. Queries traverse this model hierarchy to identify leaf blocks for scanning.

3.3.4 CUR [31]. The Cost-based Unbalanced R-tree (CUR-tree) relaxes the R-tree's strict balancing constraints so the tree shape can reflect expected node access frequency. CUR uses a workload-derived cost heuristic for data points. During insertion, it evaluates local reorganizations, including splits, promotions, and demotions, and keeps changes that lower expected query cost. This adapts the tree to workload patterns. This often improves performance in in-memory settings where computation and cache behavior dominate.

3.3.5 RWtree [7]. The RWtree extends R*tree with workload awareness. It augments top-down insertion and splitting with a learned cost model that estimates the trade-off of placing new entries in each child node. During insertion, it predicts the subtree or split strategy that best reduces future query cost, often favoring layouts aligned with high-demand regions. This reduces MBR overlap in frequently queried regions and lowers query cost.

3.3.6 Implementation. Our implementation of R*tree uses the node selection mechanism described in [3]. For RWtree, we implement a simplified learned cost model following [7]. We encode each candidate MBR, either an existing node MBR or an expanded-node MBR, as a four-dimensional point using its lower and upper bounds, and use a learned density estimator [36] to estimate query-overlap cost. During insertion, RWtree selects the subtree whose expanded MBR yields the smallest predicted overlap-cost increase

for the new point. STR [16] partitions the data into equal-sized subsets by alternately splitting along each dimension at each level of the tree. CUR [31] extends STR by replacing equal-sized splits with query-weighted splits, where partitions contain an equal estimated query cost rather than an equal number of points. We compute query-weighted splits by weighting each point by the number of queries that access it and binary-searching cumulative weights for split locations. The Rsmi index is implemented in Python using PyTorch, following the construction procedure outlined in [29]. After construction, we store the learned data partitions (the bounding boxes of each partition) and the data points within each leaf-level node.

3.4 Ordering-Based Indexes

Ordering-Based Indexes linearize spatial coordinates into a one-dimensional order. This makes standard one-dimensional indexes applicable while preserving spatial relationships through space-filling curves or learned mappings.

3.4.1 Zindex [2, 23, 34]. Zindex uses a Z-order space-filling curve to linearize points, then indexes the resulting one-dimensional values with a structure such as a B-tree.

3.4.2 ZMindex [35]. ZMindex extends Zindex by replacing the B-tree with a Recursive Model Index [14]. ZMindex combines the compactness of learned indexes with the ordering efficiency of space-filling curves.

3.4.3 WAZI [28]. WAZI learns a workload-aware space-filling curve based on Z-order. WAZI uses learned density estimates and a Z-order cost function to optimize the layout for query processing. It also exploits Z-order monotonicity to skip large data ranges irrelevant to a query.

3.4.4 Implementation. We use the publicly available implementations of Zindex and WAZI from [28]. For ZMindex [35], we adapt the original design to our paged architecture: after sorting by Z-order, we group points into fixed-size blocks, store each block's minimum and maximum Z-order values, and index those ranges with a PGM-index [8], replacing the original RMI [14].

4 Scope of Our Study

In this section, we outline the key questions that guide our experimental evaluation, the context in which this evaluation is performed, and the experimental parameters considered. To set the stage, we first present the motivation behind our research questions and provide an overview of our experimental plan. We then describe the synthetic data generation process used to construct datasets and query workloads for our index evaluation. Finally, we provide a high-level summary of index implementation and experimental setup details, establishing a basis for the thematic sections that follow.

4.1 Research Questions

4.1.1 Block Size. We first examine the effect of block size, also known as leaf size, which is one of the most influential parameters in spatial indexing. It controls the granularity of the data layout and dictates how many points reside in each leaf node. A small block size increases the number of index nodes, requiring more query-processing steps to locate relevant blocks but reducing the number of irrelevant points examined. Conversely, a large block size reduces the number of lookup steps but risks scanning many unnecessary points. Despite its importance, most prior studies either fix block size arbitrarily or assume defaults inherited from database systems without questioning their suitability for different index families. For learned indexes, where layout decisions are closely tied to modeling assumptions, block size may interact with model accuracy and workload sensitivity in unexpected ways. Understanding these interactions is critical both for fair benchmarking and for guiding practical index tuning.

RQ1: *How does block size influence query latency across indexes, and which configurations yield optimal performance under varying query selectivities?*

We answer this research question by systematically varying block size across all index types, measuring query latencies, and identifying sweet-spot configurations. This reveals how each index family responds to block-size tuning and whether learned indexes adapt better than traditional indexes.

4.1.2 Effect of Data and Query Distribution. The advantage of learned indexes lies in their ability to adapt to the underlying data distribution. In particular, their ability to efficiently model skewed data distributions is key to improving performance, yet this aspect has not been evaluated systematically. Prior evaluations often highlight strong performance on datasets such as OSM [27] and TPC-H [33] but rarely isolate the specific role of skew in shaping outcomes. Without controlled experiments, it remains unclear whether improvements come from modeling advantages or from other factors coinciding with dataset characteristics.

In addition to data skew, query skew introduces a different dimension of performance variability that is equally important to consider. Even when data is uniformly distributed, skewed query patterns—where some regions are queried more frequently—can stress certain index regions disproportionately. This is particularly relevant for query-agnostic learned indexes, which optimize only for the data distribution. Analyzing index behavior under varying levels of query skew reveals robustness in handling workload variability, making query-skew analysis essential for evaluating indexes.

RQ2: *How do skewed data and query distributions affect the performance of learned indexes?*

We answer this research question by generating synthetic datasets and workloads with precisely controlled levels of skew using Gaussian Mixture Models. By separating the effects of data skew and query skew, we assess robustness and identify which indexes gracefully handle variability in data and query skew.

4.1.3 Refinement Latency versus Scan Latency. Every range query can be decomposed into two phases: refinement, where the index identifies potentially relevant blocks, and scanning, where those blocks are filtered to produce results. The relative cost of these phases reveals how an index achieves efficiency. Some structures minimize refinement effort by simplifying search, but at the cost of scanning many irrelevant points. Others focus on aggressive pruning, potentially incurring high refinement overhead. Surprisingly, most evaluations report only total latency, obscuring whether gains

come from refinement, scanning, or both. Without this breakdown, it is impossible to understand why one index outperforms another, or under what conditions design choices pay off.

RQ3: *How do indexes balance refinement and scan phases, and which designs optimize one over the other?*

We answer this research question by separately measuring refinement and scan latencies for each index across configurations. This breakdown reveals the two levels of optimization possible within learned indexes: modeling the data distribution to minimize refinement, and optimizing block layout to minimize scan.

4.1.4 Indexing Disk-Based Data. While in-memory experiments dominate the literature on learned indexes, many real-world systems operate under disk-based storage constraints [1]. Disk access reshapes performance dynamics: refinement remains relatively cheap, but scanning incurs orders of magnitude higher latency due to I/O bottlenecks. These conditions can shift the optimal trade-offs observed in-memory, making previously efficient designs suboptimal. Empirical evidence on learned indexes with disk-resident data is limited, with only a few published disk-based evaluations. Understanding the disk-based performance of learned indexes is crucial for assessing their practical viability.

RQ4: *How do disk-based storage conditions alter the optimal block size and the refinement-scan latency trade-off compared to in-memory settings?*

We answer this research question by repeating our experiments with data stored on disk via memory mapping and contrasting them with in-memory results. This reveals how storage medium changes the landscape of optimal configurations and which indexes are resilient across both settings.

4.1.5 Index Construction Cost and Redemption. Query performance is only one aspect of index utility. Construction cost is also critical, especially in systems where data changes frequently or indexes must be rebuilt often. Learned indexes typically require expensive training and layout optimization, raising the question of whether their query-time benefits justify their upfront construction overhead. In practice, a method that accelerates queries but takes days to build may be unsuitable except at very large scales. Yet construction costs are often reported without systematic comparison or analysis of when they "pay for themselves." Filling this gap in the literature is essential for moving from theoretical performance to practical deployment.

RQ5: *What are the construction costs of different indexes, and under what query volumes are these costs effectively amortized?*

We answer this research question by measuring the build times and calculating the "redemption" threshold, i.e., the number of queries required to offset construction overhead. This yields a practical perspective on which indexes suit short-lived tasks and which are viable only for massive workloads.

4.1.6 Index Selection. Ultimately, practitioners seek to answer a fundamental question: which index should they use for a specific dataset and workload? Yet prior evaluations typically stop short of actionable guidance, instead presenting raw performance numbers. The reality is that no single index dominates across all conditions—performance depends on skew, selectivity, and storage environment. Without a systematic selection method, users face trial-and-error or rely on intuition. A principled way of translating workload characteristics into index choice would make research insights directly usable in practice.

RQ6: *Given data and query workload statistics, which index is expected to provide the best range query performance?*

We address this question by ranking indexes across experimental configurations and distilling the observed patterns into a decision tree. This offers a practical, data-driven guide to index selection, bridging the gap between experimental evaluation and deployment.

4.1.7 Validation of Synthetic-Data Insights. Synthetic benchmarks allow us to vary skew and selectivity in a controlled manner, but they also raise an important question of external validity. The practical value of the insights derived from our synthetic analysis depends on whether they remain predictive when the underlying spatial points come from real-world data. To test this, we conduct a validation experiment that combines real OpenStreetMap point sets with synthetic query workloads generated from Gaussian mixture models. This hybrid setting preserves control over the query distribution while evaluating whether the index-selection guidance learned from synthetic data transfers to more realistic spatial layouts.

RQ7: *Do the index-selection insights learned from synthetic data generalize to real-world data distributions?*

We answer this research question by evaluating the learned decision tree on 25 validation datasets, each containing 8 million real-world OpenStreetMap points and paired with 1,000 synthetic queries drawn from Gaussian mixture models. This measures whether the selection rules extracted from controlled synthetic experiments remain useful when applied to real spatial point configurations.

Algorithm 1: Generate Data Points $\mathcal{D}$ and Query Workloads $\mathcal{Q}$

Input : number-of-clusters C, clusteredness scale
Output : A set of datasets with varying clusteredness $\mathcal{D}$ and a set of queries for each dataset $\mathcal{Q}$

ClusterMeans $\mathcal{M}_D$:= `Sample` (UNIFORM, C)
ClusterCovariances C_D:= Sample random 2D covariance matrices for each cluster.
foreach *Clusteredness scale* κ_D **do**
  $\mathcal{D}_i$ = `SampleMixGaussian` ($\mathcal{M}_D$, `Scale` (C_D, κ_D))
  `// Sample the query cluster centers from data`
  QueryClusterMeans $\mathcal{M}_Q$:= `Sample` ($\mathcal{D}_i$, C)
  QueryClusterCovariances C_Q:= Sample random 2D covariance matrices for each cluster.
  foreach *Clusteredness scale* κ_Q **do**
    $\mathcal{Q}_{i,j}$ = `SampleMixGaussian` ($\mathcal{M}_Q$, `Scale` (C_Q, κ_Q))
    $\mathcal{Q}_i$ += { $\mathcal{Q}_{i,j}$ }
  $\mathcal{D}$ += { $\mathcal{D}_i$ }
  $\mathcal{Q}$ += { $\mathcal{Q}_i$ }
Return : $\mathcal{D}$, $\mathcal{Q}$

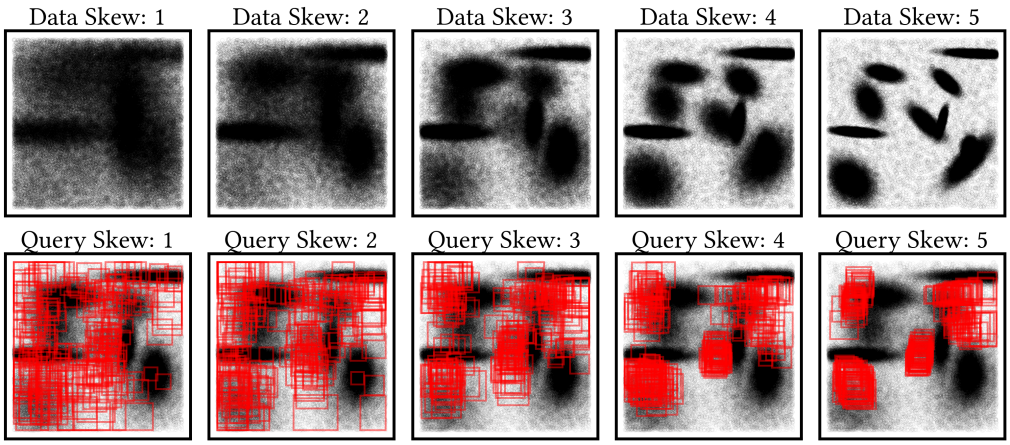


Figure 2: Example datasets and query workloads generated from Gaussian mixture models (GMMs). Top: data points with increasing levels of skew drawn from the same underlying GMM. Bottom: corresponding query distributions generated from a GMM aligned with the data points (shown here for data skew = 3).

4.2 Dataset and Query Workloads

We generate synthetic datasets and query workloads using Gaussian Mixture Models (GMMs). Our primary focus is on density-based skew, where data points or queries are unevenly distributed across the spatial domain. To quantify this skew, we use a grid-based metric called Spatial Density Entropy (SDE) [18, 19, 40]. SDE is calculated by overlaying a uniform grid over the data space, counting the number of data points within each grid cell, and computing the entropy of this distribution. The maximum SDE is achieved when all grid cells contain an equal number of points, corresponding to a uniform distribution, whereas low values indicate increasingly skewed data. To construct a dataset, we begin by randomly sampling the number of Gaussian components in the GMM. The mean of each component is sampled uniformly from the unit square, while the covariance matrices are sampled randomly to capture a range of spread patterns. From each GMM configuration, we draw sample points and discard any that fall outside the unit square. This procedure allows us to generate datasets with fine-grained control over the level and structure of clustering. To construct query workloads, we follow a similar approach to dataset generation. Specifically, we sample query centers from a GMM and expand each center into a rectangular query window to meet a target selectivity. Unlike data generation, the means of the Gaussian components for the query GMM are not sampled uniformly. Instead, they are sampled from the actual data points in the dataset, ensuring that the query distribution is correlated with the data distribution. This design mimics real-world scenarios where query patterns often align with dense or semantically important regions in the dataset, such as shopping districts in a city map or popular tourist attractions in a geographical dataset. The covariance matrices for the query GMMs are again sampled randomly to introduce variability in query shapes and orientations. To simulate a range of skew levels, we scale the covariance matrices of the Gaussian components along their principal axes. This scaling allows us to control the degree of clustering while preserving the overall structure of the GMM. From each parameterization, we generate five datasets with different levels of data skew. These datasets enable us to isolate the effect of skew while maintaining the same underlying GMM structure. Figure 2 (top) visualizes a set of such datasets generated from a shared GMM under varying skew levels. Similarly, we generate five sets of query workloads for each dataset, varying only the clustering parameter. Figure 2 (bottom) shows one example of query distributions derived from the same underlying data. This structured generation procedure allows us to independently control and analyze the effects of both data and query skew on index performance. Table 2 summarizes the parameters used in our data and query generation process. Algorithm 1 describes the process of generating both datasets and query workloads, showcasing how parameters are sampled and scaled to achieve the desired skew levels.

4.3 Implementation and Experimental Setup

The index structures evaluated in our experiments are implemented with a unified storage backend. This backend stores data as a collection of blocks, which function similarly to the leaf nodes in traditional indexing schemes. Range query processing in each index is executed in two distinct phases. The *refinement phase* involves leveraging the internal information maintained by the index to identify a relevant subset of blocks. In the subsequent *scan phase*, these selected blocks are accessed, and the contained data points are filtered to produce the final query result.

All index structures are implemented in C++ as search structures that, given a query, return a refined list of data blocks. A common block-based storage layer is used for all indexes to store and retrieve data efficiently for scanning. For experiments involving disk-based data, this block storage is backed by a memory-mapped file to simulate realistic access patterns while preserving a consistent interface and layout across index types. Specifically, we construct the index in memory and store the blocks in a memory-mapped file at the end of index construction. For indexes with learned components, we use a 10% random sample of the dataset and associated query workload for training. In the case of Rsmi, training is performed in Python using PyTorch. After the model is trained, the resulting index structure—comprising learned partition boundaries and block

Table 2: Experimental parameters; block size is optimized per index and configuration from the options listed here, and 0.1024% is the default selectivity unless varied.

Parameter	Values (default in bold)
Dataset size	8×10^6
Query selectivity (%)	[0.0064, 0.0256, **0.1024**, 0.4096, 1.6384]
Query workload size	10,000
Avg. data SDE per level	[0.985 0.975 0.958 0.924 0.870]
Avg. query SDE per level	[0.963 0.933 0.890 0.830 0.755]
Block size	[32, 64, 128, 256, 512, 1024, 2048, 4096]

assignments—is exported and used as a read-only search index in C++ for evaluation.

All experiments are conducted on a machine equipped with an Intel Xeon E5-2680v4 CPU @ 2.40GHz and 16 GB of RAM. All evaluations are single-threaded, with no GPU acceleration or parallelism used, to ensure consistency in latency measurements and to reflect typical CPU-bound deployments. We performed detailed experiments on a dataset of 8 million data points across block-size, query-selectivity, data-skew, and query-skew settings. We use these detailed experimental results to answer the research questions introduced above. Table 2 summarizes the complete set of parameters used in our experiments.

5 Experimental Results

5.1 Optimal Block Size

To address "*RQ1: How does block size influence query latency across indexes, and which configurations yield optimal performance under varying query selectivities?*", we analyze the impact of block size on query latency. By varying block sizes, we identify optimal configurations for each index and examine how these optima change with query selectivity.

Figure 3 (top) illustrates the average query latency for different index structures across varying block sizes under a workload with 0.1024% selectivity. The latencies are averaged across multiple configurations of data and query skew. Most latency curves exhibit a clear U-shaped pattern as block size changes, indicating a workload-dependent performance sweet spot. This sweet spot reflects the trade-off between higher refinement overhead for many small blocks and higher scan cost when coarse blocks contain more irrelevant points. At the default selectivity, Flood achieves the lowest tuned latency, followed closely by GridFile and KDtree. The advantage of Flood over GridFile is modest after tuning, but it is more robust to suboptimal block-size choices. QDtree remains slower than KDtree after block-size tuning, suggesting that the block-access objective used in [38] does not fully translate to lower local query latency. Among data-partitioning indexes, Rsmi provides the best tuned performance, while STR is close and CUR, RWtree, and R*tree are slower on average. This indicates that learned data partitioning can help, but only when its construction cost is paired with a layout that also controls scan overhead. The query-aware data-partitioning variants do not uniformly dominate their query-agnostic counterparts. In particular, CUR tends to choose fine blocks but pays high refinement cost, while RWtree improves over R*tree in several tuned configurations without becoming a top performer. For order-based indexes, WAZI is strongest at finer granularities, whereas ZMindex becomes competitive only when the block size is tuned carefully. This makes ZMindex the clearest example of block-size sensitivity: its tuned latency is comparable to Zindex and WAZI, but its average latency over random block sizes is much worse.

These observations indicate that the performance of index structures can vary significantly depending on the chosen block size. This suggests that using a fixed block size may introduce bias into performance analysis of indexing methods. To understand the effect of using a suboptimal block size, we compare the query latency of indexes evaluated with their respective optimal block sizes against the average latency across all considered block sizes. Figure 4 compares the query latency for indexes using optimized block sizes versus the expected latency over a random block size. The results show that both relative and absolute performance change substantially when the block-size parameter is chosen optimally. For example, ZMindex is comparable to Zindex under tuned block sizes at 0.1024% selectivity. However, averaging across all tested block sizes makes ZMindex roughly 2.8× slower than its tuned latency, making it the most sensitive method in Figure 4.

To remove the effects of block-size selection from further evaluations, we use only the results corresponding to the optimal block size for each index under each workload. This reflects the best-case performance achievable by an administrator performing index tuning. Figure 3 (bottom) plots the optimal block size as a function of query selectivity, averaged across other workload parameters. For most indexes, optimal block size increases with selectivity: highly selective queries favor finer blocks, while less selective queries tolerate coarser layouts. The exception is ZMindex, whose optimal block size remains comparatively large and does not follow the same monotonic trend. Query awareness does not imply uniformly smaller blocks: CUR and WAZI favor finer layouts than STR and Zindex, whereas Flood often uses larger average blocks than GridFile while still improving latency.

These results indicate two critical insights. First, optimizing block size is essential for a fair comparison of spatial indexes. Second, the optimal block size is itself an informative workload signal: it reveals whether an index gains more from reducing refinement cost or from reducing false-positive scans.

5.2 Effect of Data and Query Skew

Having established the role of block size, we turn to "*RQ2: How do skewed data and query distributions affect the performance of learned indexes?*". Learned indexes improve on their classical counterparts by tailoring the index for a given data and query distribution. Thus, the performance of learned indexes is influenced by the properties of data and query workloads. To assess how data and query workload skew affect index performance, we analyze the relative performance changes for each index based on these two properties. Figure 5 shows how the query latency of each index changes as we vary data and query skew. Each cell is computed as the relative percentage change in query latency for an index under a given configuration, compared with its latency under the most uniform configuration (data skew = query skew = 1, bottom left of each figure).

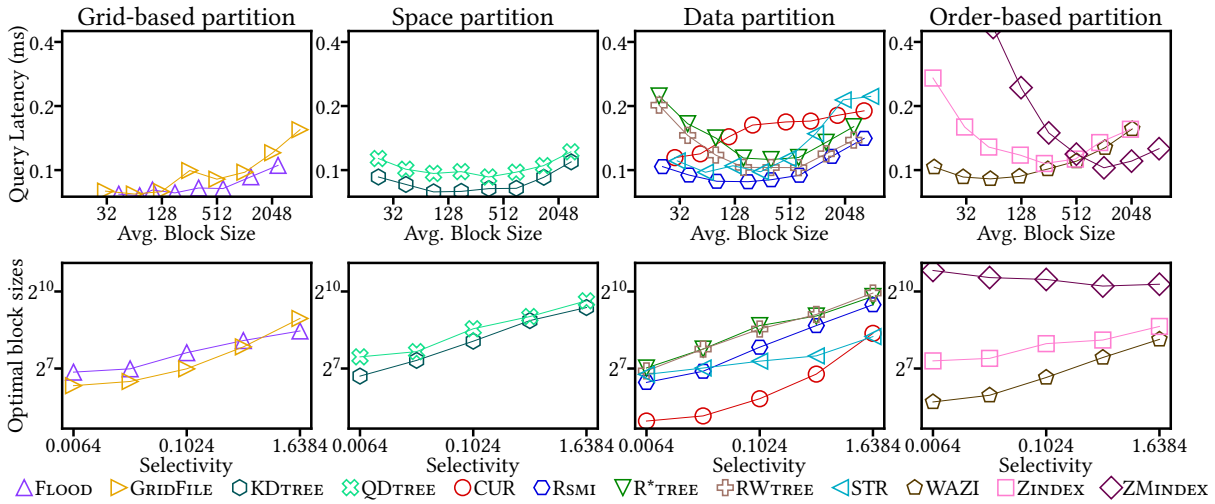


Figure 3: Query latency across block sizes (top) and optimal block size across selectivities (bottom). Realized average block size is about 73% of configured block size because several construction algorithms split data non-uniformly. The optimal block sizes are identified by local minima (sweet-spot), marking tuned granularities.

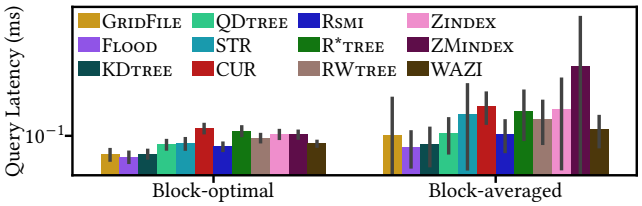


Figure 4: Query latency with optimized block sizes versus expected latency over random block sizes. Most indexes gain 1.2–1.6× from tuning, while ZMINDEX is the most sensitive at about 2.8×.

We observe two high-level trends in the figure. First, data skew has a stronger and more index-specific effect than query skew. As data skew increases, GRIDFILE, FLOOD, STR, CUR, R*TREE, RWTREE, ZINDEX, and ZMINDEX become slower on average, with ZINDEX and STR showing the largest increases. By contrast, KDTREE, QDTREE, and WAZI benefit from higher data skew, and RSMI remains comparatively stable. Second, query skew does not uniformly reduce latency. Concentrating queries can reduce work for STR, R*TREE, and QDTREE in some low-data-skew settings, but it can also increase latency for RWTREE, ZMINDEX, and CUR when hot query regions overlap dense or inefficient blocks. Overall, the magnitude of the data-skew effect is larger than the query-skew effect for most indexes.

This indicates that index performance is more sensitive to the data layout induced by skew than to query concentration alone. A skewed query workload helps only when the index layout isolates the queried dense regions into compact blocks. The new results therefore show that learned or query-aware construction is not automatically robust to skew; its benefit depends on whether the resulting partitioning reduces false-positive scans.

5.3 Refinement Latency versus Scan Latency

While skew captures workload variability, it is also critical to separate latency components. To answer "*RQ3: How do indexes balance refinement and scan phases, and which structural designs favor one over the other?*", we measure refinement and scan latencies independently. This reveals how indexes govern the balance between locating blocks and minimizing unnecessary scans.

Figure 6 plots how the average scan and refinement latencies change with increasing query selectivity. All indexes typically spend more time scanning data points than refining candidate blocks. As selectivity increases, scan latency becomes increasingly dominant: the scan-to-refinement ratio grows from single digits at the lowest selectivity to tens or hundreds at the highest selectivity for most indexes. FLOOD and GRIDFILE have very similar refinement behavior, but FLOOD achieves slightly lower scan cost and therefore the best tuned latency at the default selectivity. KDTREE is competitive because its scan cost per result point is the lowest across most selectivity levels, even though its refinement cost is higher than the grid-based indexes. QDTREE reduces refinement cost relative to KDTREE, but it scans more points and is therefore slower after tuning. This indicates that minimizing block accesses alone is insufficient when the selected blocks contain more false-positive points. RSMI is the best data-partitioning method after tuning, with lower total latency than CUR, RWTREE, and R*TREE and latency close to STR. The iterative-insertion indexes R*TREE and RWTREE do not consistently reduce scan cost enough to offset their construction and layout complexity. CUR is the clearest outlier in this family: it uses fine blocks, but its refinement overhead is high. The order-based indexes show a different trade-off. WAZI minimizes false positives, especially at higher selectivities, but KDTREE, FLOOD, and GRIDFILE still achieve lower scan time per result point. ZINDEX and ZMINDEX scan more false positives than WAZI; ZMINDEX also has the highest refinement overhead among the order-based methods. Consequently, ZMINDEX is not the high-selectivity winner in the new synthetic results despite being competitive under tuned

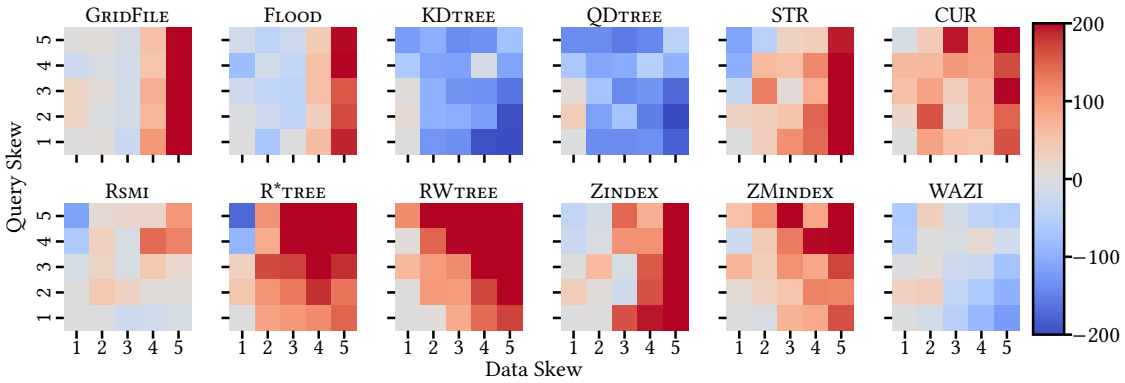


Figure 5: Percentage change in index performance across different levels of data skew (x-axis) and query skew (y-axis). The heatmap shows how each index's performance changes relative to its latency on near-uniform data and query workloads (bottom-left cell). Increasing data skew degrades GridFile, Flood, STR, CUR, R*tree, RWtree, Zindex, and ZMindex, while KDtree, QDtree, and WAZI improve or remain stable. Query skew has a smaller and more mixed effect: STR, R*tree, and QDtree benefit in several settings, while RWtree, ZMindex, and CUR can become slower.

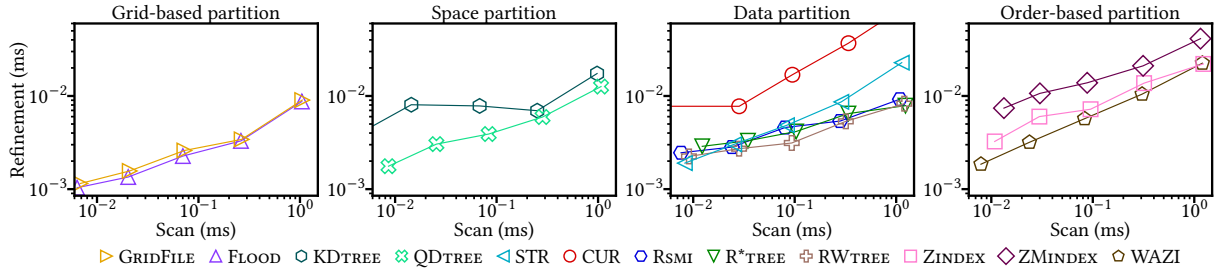


Figure 6: Refinement (y-axis) and scan (x-axis) latency across queries of increasing selectivity. Except at the smallest selectivities for a few methods, scan latency dominates refinement latency, and the gap widens rapidly as selectivity increases.

block sizes in Section 5.1. The high-selectivity advantage instead belongs to indexes that combine low scan cost per result point with controlled false positives.

Figure 7 (left) shows the changes in refinement latency per block and the percentage of blocks accessed for each index over queries of different selectivity. The left plot shows that block-access behavior is strongly family-dependent. GridFile and Flood access similar fractions of blocks and have low per-block refinement costs. KDtree and QDtree also have similar access patterns, but QDtree's selected blocks contain more irrelevant points, leading to worse scan behavior. Among data-partitioning methods, CUR pays the highest refinement overhead, whereas Rsmi and RWtree access moderate fractions of blocks with lower per-block refinement cost. Thus, the learned variants do not all behave the same: Flood and QDtree reduce refinement overhead relative to their classical counterparts, while CUR and ZMindex are refinement-heavy. The spread within each selectivity level also confirms the finding from Section 5.2: data and query skew affect how many blocks a query must refine.

The scan latency of a query is directly correlated with the number of points scanned for each index. The differences in scan latency between indexes depend on the number of false-positive points scanned during query processing. Figure 7 (right) shows the changes in scan latency per result point and false positives for each index over queries of different selectivity. We make four observations from the figure. First, WAZI consistently scans the fewest false-positive points among the order-based indexes and is often the best overall on this metric. Second, low false-positive counts do not fully determine scan latency: KDtree, Flood, and GridFile usually have the lowest scan latency per result point. Third, R*tree remains weak among data-partitioning indexes, while RWtree improves over R*tree but rarely reaches the top tier. ZMindex incurs more false positives than WAZI and combines that with higher refinement cost, explaining its sensitivity to block-size tuning. Finally, the best scan efficiency is achieved by simple layouts with low per-point scan overhead, especially KDtree, Flood, and GridFile.

Across all indexes, scan latency dominates once selectivity grows, but the source of scan efficiency differs by family. WAZI is strongest at minimizing false positives, whereas KDtree, Flood, and GridFile provide the lowest scan latency per result point. In contrast, R*tree, CUR, and ZMindex show that sophisticated partitioning can still lose when it creates either high refinement overhead or high false-positive scans.

5.4 Considerations of Indexing Disk-Based Data

The previous analysis assumed in-memory configurations. To address "*RQ4: How do disk-based storage conditions alter the optimal block size and the refinement-scan latency trade-off compared to in-memory settings?*", we extend evaluation to disk-backed data. This

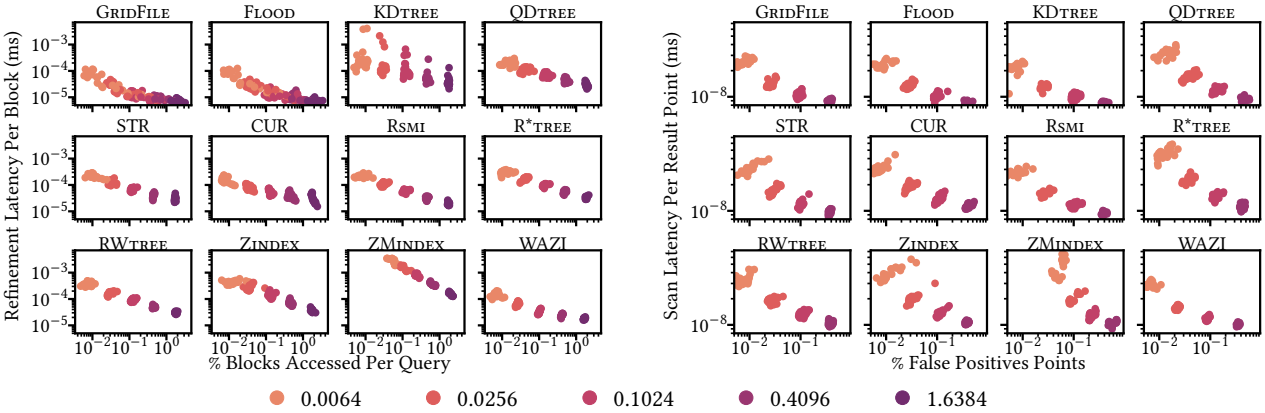


Figure 7: Left: refinement latency per block versus blocks accessed. Right: scan latency per result point versus false-positive points scanned. Color encodes query selectivity. False positives and scan latency do not perfectly align: WAZI minimizes false positives, while KDTree, Flood, and GridFile scan fastest.

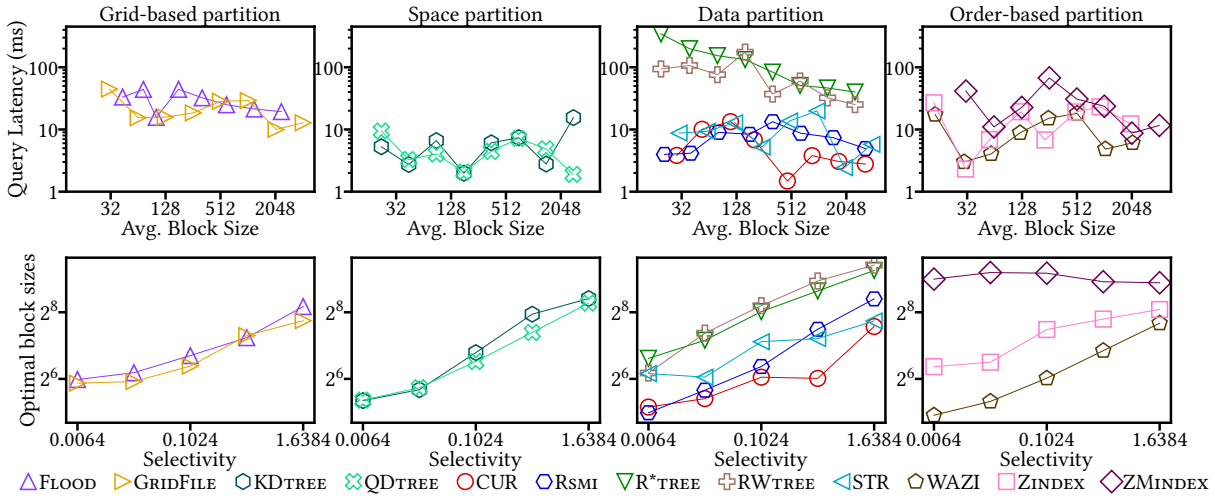


Figure 8: Performance comparison of indexes on disk-backed data pages. Very fine blocks often incur high disk-backed latency, but the best block size remains index- and workload-dependent rather than simply favoring the coarsest layout. Optimal block size generally still increases with query selectivity, although the disk-backed optima are not uniformly larger than the in-memory optima.

highlights how slower access costs shift block size preferences and alter refinement-to-scan trade-offs.

The optimal block sizes presented in Section 5.1 identify the sweet spot between added refinement cost (more index layers and smaller block sizes) and reduced scan time from smaller block sizes. This trade-off is determined by the latency of data access during scanning, which remains nearly constant in in-memory settings but changes substantially on disk due to slower access. To analyze this effect, we store data blocks in a memory-mapped file and evaluate index performance across all data and query configurations under disk-backed conditions.

Our experiments show that the effective data-access cost of disk-backed scans is about 3.6× higher than in-memory scans, as shown in Figure 9. We call this the effective data-access cost because the actual cost of accessing a block from disk is at least 100× higher than accessing a similar block from memory. The access cost is lowered because the operating system caches recently accessed blocks, reducing measured latency. Figure 8 (top) illustrates the average query latency for different index structures across varying block sizes under a workload with 0.1024% selectivity. Unlike the in-memory results, the disk-backed curves are less uniformly U-shaped. Very small blocks can be expensive because they require many page fetches, but increasing block size eventually raises scan cost; the balance point differs by index family. Figure 8 (bottom) shows that optimal block size generally increases with selectivity, but the absolute values are often smaller than their in-memory counterparts because disk-backed evaluation penalizes excessive scans. At the default selectivity, the fastest disk-backed methods are KDTree, Rsmi, WAZI, and QDTree, indicating that structural differences still matter under higher access cost. Overall, these

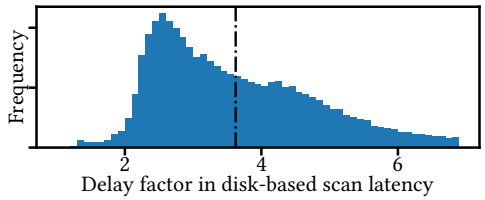


Figure 9: Histogram of latency-change factor between disk-backed and in-memory configurations. On average, scanning disk-backed data was about 3.6× (dotted line) slower.

results demonstrate that the disk-backed cost model differs qualitatively from the in-memory setting: it penalizes both excessive block fetches and excessive false-positive scans.

We identify two key insights from these results. First, disk-backed performance cannot be predicted from block size alone: Flood is faster than GridFile at the default selectivity, but the relative block sizes of the two indexes vary across workloads. Second, query selectivity remains the strongest driver of the optimal granularity, while the index structure determines whether that granularity translates into low scan cost.

5.5 Index Construction Cost

Beyond latency, index adoption also depends on relative construction overhead. In response to "*RQ5: What are the construction costs of different indexes, and under what query volumes are these costs effectively amortized?*", we measure build times and compute redemption thresholds. These results indicate when higher upfront construction cost is offset by long-term query savings.

Table 3 reports build times for all indexes. We observe that learned indexes generally increase construction cost, but the magnitude varies substantially by family. STR and GridFile have the lowest construction times, as their iterative sort-based algorithms have log-linear complexity. GridFile sorts the data twice, while STR repeatedly sorts until blocks of the required granularity are achieved. The layout optimization in Flood is expensive, taking about 16× longer than GridFile, whereas QDTree is only about 1.5× slower to build than KDTree. Among data-partitioning indexes, Rsmi is the clear construction-cost outlier, while CUR and RWTree add only moderate overhead relative to STR and R*tree. Order-based learned methods remain comparatively cheap: ZMIndex and WAZI are constant-factor slower than Zindex.

We are also interested in the cost redemption of each index. We define cost redemption, measured relative to the query-aware CUR baseline, as the number of queries for an index to amortize its construction cost: $redem = \mathcal{I}^{+/-} \cdot \frac{index\ build\ time - base\ build\ time}{base\ query\ latency - index\ query\ latency}$, where $\mathcal{I}^{+/-}$ sets the table sign. In Table 3, (+) denotes eventual redemption and (-) no long-run redemption; the attached number is the crossover query count, and a missing number means the index is always or never redeemed.

We observe two key findings from the redemption behavior. First, in the in-memory setting, all indexes except R*tree eventually redeem their construction cost relative to CUR. The redemption threshold, however, varies widely: KDTree, QDTree, and ZMIndex redeem within fewer than half a million queries, whereas Rsmi requires roughly 790M queries because of its very high build time. Second, disk-backed redemption changes the picture: KDTree, QDTree, Rsmi, and WAZI redeem their construction costs, while Flood, R*tree, RWTree, Zindex, and ZMIndex do not. Low-build-time indexes such as GridFile and STR can still be preferable for small disk-backed query volumes, but their slower disk query latency prevents long-run redemption.

5.6 Index Selection

Query performance varies widely across workloads. In our experiments, four factors primarily drive these differences: data skew, query skew, storage medium, and selectivity. We therefore answer "*RQ6: Given data and query workload statistics, which index is expected to provide the best range query performance?*" by learning a simple decision tree that maps these factors to an index choice.

To construct the decision tree, we rank indexes by query latency for each data sample and query workload. For each measurable configuration, defined by data skew, query skew, selectivity, and storage medium, we retain the top-three indexes as candidate labels, so the classifier learns regions where an index is competitive rather than only where it is the single fastest method. The resulting simplified tree in Figure 10 predicts at least one top-three candidate with 57.8% overall top-three accuracy.

In the in-memory setting, Flood, KDTree, and GridFile account for nearly all top-three outcomes. Flood dominates low and medium selectivities, while KDTree becomes increasingly competitive at high selectivity; GridFile remains a strong baseline across the range. For disk-backed data, the top-three set is broader: KDTree appears most often, followed by Rsmi, Flood, GridFile, and WAZI. At high disk-backed selectivity, Rsmi, Flood, KDTree, and GridFile are most frequent among the top-three choices, while WAZI and QDTree are useful in more specialized regions.

5.7 Validation of Synthetic-Data Insights

To answer "*RQ7: Do the index-selection insights learned from synthetic data generalize to real-world data distributions?*", we evaluate the decision tree derived from the synthetic experiments on held-out real-world validation settings. Each validation instance contains 8 million two-dimensional point locations sampled from OpenStreetMap, providing spatial layouts generated by real geographic processes rather than by the synthetic GMM data generator. We pair each OpenStreetMap point set with 1,000 synthetic Gaussian-mixture queries with randomly selected query-skew levels and selectivities, testing whether the selection rules learned from synthetic data transfer to real spatial point distributions.

To validate the synthetic-study results, we analyze how effectively the decision tree selects the best-performing indexes on real-world spatial data. We generate 25 validation datasets from OpenStreetMap, each with 8 million points, and pair each dataset with 1,000 synthetic queries. For each dataset–workload pair, we build every index separately using the block-size settings identified by the synthetic experiments for the in-memory and disk-backed cases. For practical index selection, the relevant question is not only whether the tree names the exact fastest index, but how much

Table 3: Build times of each index (seconds) and their redemption relative to the CUR index (gray). Redemption indicates the number of queries (in millions) needed for an index to amortize its build cost compared with CUR.

Index	GridFile	Flood	KDtree	QDtree	STR	CUR	Rsmi	R*tree	RWtree	Zindex	ZMindex	WAZI
Build Time (sec)	9.4	148.6	16.4	24.1	6.5	13.6	2031.2	38.1	44.6	19.8	23.5	38.2
Redemption (in-memory)	(+)	(+)8.9M	(+)0.15M	(+)0.44M	(+)		(+)790M	(-)	(+)21M	(+)15M	(+)0.48M	(+)2.6M
Redemption (disk-based)	(-)0.26M	(-)	(+)1.2M	(+)0.2M	(-)0.13M		(+)4.1M	(-)	(-)	(-)	(-)	(+)0.29M

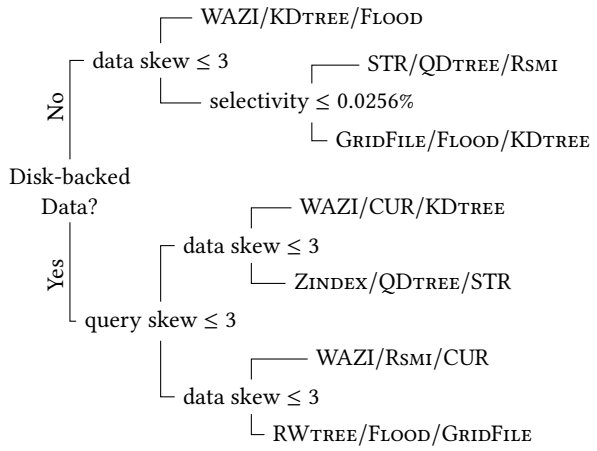


Figure 10: Decision tree classifier trained to select one of the top-performing indexes based on data skew, query skew, selectivity, and storage medium. The tree predicts at least one top-three index with 57.8% overall top-three accuracy.

latency is lost by following its recommendations. We therefore measure top-three decision regret: the latency of the best index among the tree's three recommendations divided by the latency of the empirical optimum. The in-memory results show strong transfer: the decision tree places the best-performing index in its top-three recommendations for 72% of the OpenStreetMap scenarios, closely matching its 70.9% in-memory top-three accuracy on the synthetic configurations. More importantly, the median top-three decision regret is 1.0× in memory, and the best recommended index is within 1.25× of the observed optimum in every in-memory validation scenario. Disk-backed validation is harder because block-layout errors are amplified by I/O cost. Although the top-three hit rate drops to 48%, the recommendations still have low practical cost: the median top-three decision regret is only 1.008×, and in 84% of scenarios the top-three set includes an index within 1.25× of the best disk-backed latency.

These results show that the controlled synthetic study captures transferable structure in the index-selection problem: the learned rules do not merely fit synthetic distributions, but provide actionable recommendations that typically recover the best index or one with nearly identical latency on real OpenStreetMap layouts.

6 Conclusions

This study shows that learned spatial indexes can outperform traditional indexes, but their advantage depends on specific conditions such as data characteristics and workload. Using a unified benchmarking framework, we compared multiple learned spatial indexes against established spatial indexing techniques. By optimizing index granularity for each workload, we evaluated the indexes near their strongest observed configurations and avoided conclusions that depend on a single fixed block size. The results demonstrate that learned spatial indexes are effective, although their advantage is conditional rather than universal. Block-size tuning emerges as a primary factor; most indexes improve substantially after tuning, and the optimal granularity varies with selectivity, mediating the trade-off between refinement overhead and false-positive scans. Across synthetic workloads, learned and query-aware layouts are most effective when they reduce scan costs without incurring excessive refinement costs. This finding accounts for the consistent in-memory performance of Flood, KDtree, and GridFile, the prominence of Rsmi, WAZI, and QDtree under disk-backed evaluation, and the underperformance of methods with sophisticated construction when their block layout results in numerous false positives. The experiments also show that data skew is often more consequential than query skew, and that disk-backed storage changes the cost model by penalizing both excessive page accesses and unnecessary scans. Construction costs further qualify these findings; several learned indexes amortize their build overhead only after sufficient query volume, indicating that deployment decisions should account for both query latency and amortization. These findings are synthesized into a decision tree that uses workload statistics to recommend competitive indexes. On real OpenStreetMap validation workloads, exact winner recovery is imperfect, especially on disk-backed data, but the resulting decision regret is minimal: the median top-three regret is 1.0× in memory and 1.008× on disk-backed data. This means that the decision tree usually recommends either the best index or one with nearly identical latency, making the learned selection rules useful as practical guidance. Collectively, these results underscore the value of this study as both an empirical benchmark and a practical guide for selecting spatial indexes under explicit workload and storage assumptions.

Limitations and Future Work. Synthetic datasets allowed precise control of data and query skew, but broader validation should include diverse real-world datasets. Our analysis focuses on static data; dynamic environments [10], including adaptive retraining [12] and incremental learning, remain important extensions. Beyond the representative methods studied here, further variations and hybrid designs need systematic exploration to better understand trade-offs between complexity, training cost, and efficiency. Expanding the framework to support query types beyond range queries, such as nearest-neighbor search [39] and spatial joins, would broaden its practical scope. Because we simplified some implementations for comparability, validating results against production-grade systems and heterogeneous computing environments is an important next step.

Artifacts

The code developed for this work is publicly available at https://version.helsinki.fi/ads/evaluating-learned-spatial-indexes/. The repository contains the Python script needed for Rsmi training and data generation, C++ implementations of all evaluated indexes, experiment configuration files, and scripts used to produce the reported measurements. Its README describes the required dependencies and setup process, provides a small smoke test for checking that the pipeline builds and runs locally, and gives the commands for reproducing the full synthetic experiments and for running the same workflow through Slurm. Real-workload preprocessing instructions are also included, so the artifact documents both quick validation and full experimental reproduction.

References

[1] Hussam Abu-Libdeh, Deniz Altinbüken, Alex Beutel, Ed H. Chi, Lyric Doshi, Tim Kraska, Xiaozhou Li, Andy Ly, and Christopher Olston. 2020. Learned Indexes for a Google-scale Disk-based Database. *CoRR* abs/2012.12501 (2020). arXiv:2012.12501 https://arxiv.org/abs/2012.12501

[2] Rudolf Bayer. 1997. The Universal B-Tree for Multidimensional Indexing: General Concepts. In *Worldwide Computing and Its Applications, International Conference, WWCA '97, Tsukuba, Japan, March 10-11, 1997, Proceedings (Lecture Notes in Computer Science, Vol. 1274)*, Takashi Masuda, Yoshifumi Masunaga, and Michiharu Tsukamoto (Eds.). Springer, 198–209. doi:10.1007/3-540-63343-X_48

[3] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R*-Tree: An Efficient and Robust Access Method for Points and Rectangles. In *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data, Atlantic City, NJ, USA, May 23-25, 1990*, Hector Garcia-Molina and H. V. Jagadish (Eds.). ACM Press, 322–331. doi:10.1145/93597.98741

[4] Norbert Beckmann and Bernhard Seeger. 2009. A Revised R*-Tree in Comparison with Related Index Structures. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data*. ACM, Providence Rhode Island USA, 799–812. doi:10.1145/1559845.1559929

[5] Jon Louis Bentley. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Commun. ACM* 18, 9 (Sept. 1975), 509–517. doi:10.1145/361002.361007

[6] Minguk Choi, Seehwan Yoo, and Jongmoo Choi. 2024. Can Learned Indexes Be Built Efficiently? A Deep Dive into Sampling Trade-offs. *Proceedings of the ACM on Management of Data* 2, 3 (May 2024), 1–25. doi:10.1145/3654919

[7] Haowen Dong, Chengliang Chai, Yuyu Luo, Jiabin Liu, Jianhua Feng, and Chaoqun Zhan. 2022. RW-Tree: A Learned Workload-aware Framework for R-tree Construction. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 2073–2085. doi:10.1109/ICDE53745.2022.00201

[8] Paolo Ferragina and Giorgio Vinciguerra. 2020. The PGM-index: A Fully-Dynamic Compressed Learned Index with Provable Worst-Case Bounds. *Proceedings of the VLDB Endowment* 13, 8 (April 2020), 1162–1175. doi:10.14778/3389133.3389135

[9] Jiake Ge, Boyu Shi, Yanfeng Chai, Yuanhui Luo, Yunda Guo, Yinxuan He, and Yunpeng Chai. 2023. Cutting Learned Index into Pieces: An In-depth Inquiry into Updatable Learned Indexes. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 315–327. doi:10.1109/ICDE55515.2023.00031

[10] Tu Gu, Kaiyu Feng, Gao Cong, Cheng Long, Zheng Wang, and Sheng Wang. 2023. The RLR-Tree: A Reinforcement Learning Based R-Tree for Spatial Data. *Proc. ACM Manag. Data* 1, 1, Article 63 (May 2023), 26 pages. doi:10.1145/3588917

[11] Antonin Guttman. 1984. R-Trees: A Dynamic Index Structure for Spatial Searching. In *SIGMOD'84, Proceedings of Annual Meeting, Boston, Massachusetts, USA, June 18-21, 1984*, Beatrice Yormark (Ed.). ACM Press, 47–57. doi:10.1145/602259.602266

[12] Fuma Hidaka and Yusuke Matsui. 2024. FlexFlood: Efficiently Updatable Learned Multi-dimensional Index. *CoRR* abs/2411.09205 (2024). arXiv:2411.09205 doi:10.48550/ARXIV.2411.09205

[13] Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. 2019. SOSD: A Benchmark for Learned Indexes. *CoRR* abs/1911.13014 (2019). arXiv:1911.13014 http://arxiv.org/abs/1911.13014

[14] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 489–504. doi:10.1145/3183713.3196909

[15] Hai Lan, Zhifeng Bao, J. Shane Culpepper, and Renata Borovica-Gajic. 2023. Updatable Learned Indexes Meet Disk-Resident DBMS - From Evaluations to Design Choices. *Proceedings of the ACM on Management of Data* 1, 2 (June 2023), 1–22. doi:10.1145/3589284

[16] S.T. Leutenegger, M.A. Lopez, and J. Edgington. 1997. STR: A Simple and Efficient Algorithm for R-tree Packing. In *Proceedings 13th International Conference on Data Engineering*. 497–506. doi:10.1109/ICDE.1997.582015

[17] Pengfei Li, Hua Lu, Qian Zheng, Long Yang, and Gang Pan. 2020. LISA: A Learned Index Structure for Spatial Data. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo (Eds.). ACM, 2119–2133. doi:10.1145/3318464.3389703

[18] Lingbo Liu, Zhenghong Peng, Hao Wu, Hongzan Jiao, Yang Yu, and Jie Zhao. 2018. Fast Identification of Urban Sprawl Based on K-Means Clustering with Population Density and Local Spatial Entropy. *Sustainability* 10, 8 (July 2018), 2683. doi:10.3390/su10082683

[19] Pingping Liu, Zhuang Miao, Huili Guo, Yeran Wang, and Ni Ai. 2018. Adding Spatial Distribution Clue to Aggregated Vector in Image Retrieval. *EURASIP Journal on Image and Video Processing* 2018, 1 (Feb. 2018), 9. doi:10.1186/s13640-018-0247-0

[20] Qiyu Liu, Maocheng Li, Yuxiang Zeng, Yanyan Shen, and Lei Chen. 2025. How Good Are Multi-Dimensional Learned Indexes? An Experimental Survey. *The VLDB Journal* 34, 2 (Jan. 2025), 17. doi:10.1007/s00778-024-00893-6

[21] Marcel Maltry and Jens Dittrich. 2022. A Critical Analysis of Recursive Model Indexes. *Proceedings of the VLDB Endowment* 15, 5 (Jan. 2022), 1079–1091. doi:10.14778/3510397.3510405

[22] Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. 2020. Benchmarking Learned Indexes. *Proceedings of the VLDB Endowment* 14, 1 (Sept. 2020), 1–13. arXiv:2006.12804 doi:10.14778/3421424.3421425

[23] Volker Markl. 2000. Mistral - Processing Relational Queries using a Multidimensional Access Technique. *Datenbank Rundbr.* 26 (2000), 24–25.

[24] Vikram Nathan, Jialin Ding, Mohammad Alizadeh, and Tim Kraska. 2020. Learning Multi-Dimensional Indexes. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo (Eds.). ACM, 985–1000. doi:10.1145/3318464.3380579

[25] Mathias Neufang, Emma Pajak, Damien van de Berg, Ye Seol Lee, and Ehecatl Antonio del Rio Chanona. 2024. Surrogate-Based Optimization Techniques for Process Systems Engineering. arXiv:2412.13948 doi:10.48550/arXiv.2412.13948

[26] Jürg Nievergelt, Hans Hinterberger, and Kenneth C. Sevcik. 1981. The Grid File: An Adaptable, Symmetric Multi-Key File Structure. In *Trends in Information Processing Systems, 3rd Conference of the European Cooperation in Informatics, Munich, Germany, October 20-22, 1981, Proceedings (Lecture Notes in Computer Science, Vol. 123)*, A. J. W. Duijvestijn and Peter C. Lockemann (Eds.). Springer, 236–251. doi:10.1007/3-540-10885-8_45

[27] OpenStreetMap contributors. 2025. OpenStreetMap Planet Dump. https://www.openstreetmap.org.

[28] Sachith Pai, Michael Mathioudakis, and Yanhao Wang. 2024. WaZI: A Learned and Workload-aware Z-Index. In *Proceedings 27th International Conference on Extending Database Technology, EDBT 2024, Paestum, Italy, March 25 - March 28*, Letizia Tanca, Qiong Luo, Giuseppe Polese, Loredana Caruccio, Xavier Oriol, and Donatella Firmani (Eds.). OpenProceedings.org, 559–571. doi:10.48786/EDBT.2024.48

[29] Jianzhong Qi, Guanli Liu, Christian S. Jensen, and Lars Kulik. 2020. Effectively Learning Spatial Indices. *Proceedings of the VLDB Endowment* 13, 12 (Aug. 2020), 2341–2354. doi:10.14778/3407790.3407829

[30] Nestor V. Queipo, Raphael T. Haftka, Wei Shyy, Tushar Goel, Rajkumar Vaidyanathan, and P. Kevin Tucker. 2005. Surrogate-Based Analysis and Optimization. *Progress in Aerospace Sciences* 41, 1 (Jan. 2005), 1–28. doi:10.1016/j.paerosci.2005.02.001

[31] K.A. Ross, I. Sitzmann, and P.J. Stuckey. 2001. Cost-Based Unbalanced R-trees. In *Proceedings Thirteenth International Conference on Scientific and Statistical Database Management. SSDBM 2001*. 203–212. doi:10.1109/SSDM.2001.938552

[32] Zhaoyan Sun, Xuanhe Zhou, and Guoliang Li. 2023. Learned Index: A Comprehensive Experimental Evaluation. *Proc. VLDB Endow.* 16, 8 (2023), 1992–2004. doi:10.14778/3594512.3594528

[33] TPC, Transaction Processing Performance Council. 2025. TPC-H: Decision Support Benchmark. https://www.tpc.org/tpch/.

[34] Herbert Tropf and Helmut Herzog. 1981. Multidimensional Range Search in Dynamically Balanced Trees. *Angewandte Info.* 2 (1981), 71–77.

[35] Haixin Wang, Xiaoyi Fu, Jianliang Xu, and Hua Lu. 2019. Learned Index for Spatial Queries. In *2019 20th IEEE International Conference on Mobile Data Management (MDM)*. 569–574. doi:10.1109/MDM.2019.00121

[36] Hongwei Wen and Hanyuan Hang. 2022. Random Forest Density Estimation. In *Proceedings of the 39th International Conference on Machine Learning*. PMLR, 23701–23722.

[37] Chaichon Wongkham, Baotong Lu, Chris Liu, Zhicong Zhong, Eric Lo, and Tianzheng Wang. 2022. Are Updatable Learned Indexes Ready? *Proceedings of*

the VLDB Endowment 15, 11 (July 2022), 3004–3017. arXiv:2207.02900 doi:10.14778/3551793.3551848

[38] Zongheng Yang, Badrish Chandramouli, Chi Wang, Johannes Gehrke, Yinan Li, Umar Farooq Minhas, Per-Åke Larson, Donald Kossmann, and Rajeev Acharya. 2020. Qd-Tree: Learning Data Layouts for Big Data Analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, Portland OR USA, 193–208. doi:10.1145/3318464.3389770

[39] Peng Yongxin, Zhou Wei, Zhang Lin, and Du Hongle. 2020. A Study of Learned KD Tree Based on Learned Index. In *2020 International Conference on Networking and Network Applications (NaNA)*. 355–360. doi:10.1109/NaNA51271.2020.00067

[40] Jiaoyi Zhang and Yihan Gao. 2022. CARMI: A Cache-Aware Learned Index with a Cost-Based Construction Algorithm. *Proceedings of the VLDB Endowment* 15, 11 (July 2022), 2679–2691. doi:10.14778/3551793.3551823